

Reporte de Evaluación Parcial 2 - Julio

Evaluación General

La entrega es sobresaliente y destaca como una de las más sólidas de la cohorte. El proyecto logró evolucionar una maquetación estática hacia una aplicación funcional con un stack moderno (TypeScript, Express, Zod, Supabase) que excede el nivel mínimo esperado para este bloque. La arquitectura de la API muestra un entendimiento maduro de cómo separar responsabilidades (controladores, servicios, repositorios), y la integración full-stack es real y robusta. Es un trabajo que se percibe y opera como un producto profesional.

Fortalezas

- Modelado de Persistencia:** Esquema SQL impecable en PostgreSQL, utilizando UUIDs, restricciones `CHECK` para estados lógicos, y habilitación de Row Level Security (RLS).
- Arquitectura de Backend:** Separación modular clara (Domain-Driven Design) y uso de TypeScript para asegurar contratos de datos, una práctica muy superior al promedio de segundo año.
- Defensa en Profundidad:** Implementación de CORS (a través de validación estricta de orígenes con variables de entorno), Rate Limiting, y validación estricta de esquemas de entrada con Zod antes de interactuar con la base de datos.
- Integración UX/API:** El frontend consume la API asíncronamente para validar la disponibilidad en tiempo real antes de permitir el envío de datos, mejorando drásticamente la experiencia de usuario.
- Orden de Proyecto:** Repositorio limpio, con variables de entorno bien documentadas (`.env.example`) y scripts de ejecución claros.

Aspectos a Mejorar / Recomendaciones de Escalabilidad

- Manejo de Secretos:** Aunque la entrega es muy sólida, para futuros pasos hacia la puesta en producción real, es vital asegurar que las claves de "Service Role" de bases de datos como Supabase nunca compartan espacio o riesgo de exposición con el entorno cliente.
- Transaccionalidad (TCL):** A medida que la aplicación crezca (por ejemplo, manejando usuarios y reservas como entidades separadas pero vinculadas en un solo flujo), será necesario aplicar transacciones SQL explícitas (`BEGIN` , `COMMIT` , `ROLLBACK`) para garantizar que operaciones complejas no queden a medias en caso de un error de red.

Desglose por Rúbrica

- Modelado y Persistencia: 25/25
- Arquitectura de API: 25/25
- Frontend y UX: 20/20
- Integración Full-Stack: 15/15
- Robustez y Validación: 10/10
- Orden y Git: 5/5

Puntos Extra (+): Capa de seguridad implementada en servidor (CORS por origen y Rate Limiting) y base de datos (RLS).

Resultado Final

- Puntaje: 100/100
- Nota: 7,0

Síntesis Técnica

El trabajo demuestra no solo un entendimiento sobre cómo hacer que el código funcione, sino cómo se construye software real, escalable y seguro. El salto desde la primera entrega estática a este ecosistema integrado es notable y refleja un criterio técnico muy maduro para el nivel de la carrera. La base técnica está consolidada; el siguiente salto natural es escalar la arquitectura hacia flujos de despliegue continuo y patrones aún más complejos.